šolski center ptuj
elektro in računalniška šola

Volkmerjeva cesta 19, 2250 Ptuj

Spletna aplikacija za upravljanje virtualnih računalnikov

Mentor: Marjan Čeh

Avtor: Nik Mlakar, 4.a

Program: SSI, tehnik računalništva

Ptuj, Marec 2026

POVZETEK

Glavna tema moje naloge je bila izdelava spletne aplikacije za upravljanje virtualnih računalnikov. Namen naloge je bil razviti preprosto aplikacijo za nadzor virtualnih računalnikov preko spletnega brskalnika. Za to temo sem se odločil, zaradi zanimanja za virtualizacijo in predhodnega znanja o izdelavi spletnih aplikacij.

V tej maturitetni nalogi bom predstavil osnove o virtualizaciji, postopke in metode, ki sem jih uporabil, ter funkcije ki jih aplikacija omogoča.

Ključne besede: virtualizacija, spletna aplikacija, virtualni računalniki, React, Flask, libvirt

SUMMARY

The main topic of my thesis was the creation of a web application for managing virtual computers. The purpose of the thesis was to develop a simple application for controlling virtual computers via a web browser. I chose this topic due to interest in virtualization and prior knowledge of building web applications.

In this graduation thesis, I will present the basics of virtualization, the procedures and methods I used, and the functions of the application.

Keywords: virtualization, web application, virtual machines, React, Flask, libvirt

KAZALO

1 UVOD....1
2 VIRTUALNI RAČUNALNIKI....2
2.1 ZGODOVINA VIRTUALNIH RAČUNALNIKOV....2
2.2 AKTUALNOST VIRTUALNIH RAČUNALNIKOV....2
3 POTEK DELA....3
3.1 RAZVOJ UPORABNIŠKEGA DELA....5
3.1.1 PODATKOVNA STRUKTURA....5
3.1.2 DELOVANJE....5
3.1.3 UPRAVLJANJE NAPAK....6
3.2 RAZVOJ APLIKACIJSKEGA DELA....7
3.2.1 PODATKOVNA STRUKTURA....7
3.2.2 POVEZAVA Z BAZO....8
3.2.3 UPRAVLJANJE NAPAK....10
3.2.4 ZAGON APLIKACIJE....11
4 Funkcije....12
4.1 FUNKCIJE VEZANE NA VIRTUALNE RAČUNALNIKE....12
4.2 FUNKCIJE VEZANE NA ZBIRKO PODATKOV....13
4.3 FUNKCIJE VEZANE NA OMREŽJE....13
5 ZAKLJUČEK....14
6 VIRI IN LITERATURA....15

Kazalo slik

Slika 1: Prikaz poteka delovanja aplikacije....4
Slika 2: Podatkovna struktura uporabniskega dela....5
Slika 3: Primer kode za zahtevo podatkov....6
Slika 4: Podatkovna struktura aplikacijskega dela....7
Slika 5: Primer implementacije SQLite....9
Slika 6: Primer flask kode....10
Slika 7: Primer zagona aplikacije....11

1 UVOD

Virtualizacija je pomemben del sodobnih informacijskih sistemov, saj omogoča zagon več navideznih računalnikov na eni fizični napravi.

Namen te naloge je razviti spletno aplikacijo za upravljanje virtualnih računalnikov, ki bi uporabnikom omogočala enostavno in pregledno uporabo.

Za izdelavo aplikacije sem uporabil knjižnico React za razvoj uporabniškega vmesnika, ogrodje Flask za strežniški del aplikacije ter orodje libvirt za upravljanje virtualnih računalnikov.

Cilj naloge je izdelati delujočo spletno aplikacijo, ki omogoča enostaven in pregelden nadzor virtulnih računalnikov. Aplikacija je namenjena predvsem Linux uporabnikom, ki želijo preprosto rešitev za upravljanje virtualnih okolij.

2 VIRTUALNI RAČUNALNIKI

Virtualni računalniki so v tehnološkem svetu ključnega pomena, saj omogočajo izvajanje več operacijskih sistemov na eni fizični napravi. Namesto da bi bil računalnik omejen na en sam sistem, lahko z uporabo virtualizacije ustvarimo več virtualnih okolij, ki delujejo neodvisno drug od drugega.

Osnova virtualnih računalnikov je monitor virtualnega računalnika oziroma hipervizor(hypervisor). Namen te programske opreme je kreiranje in zagon virtualnih računalnikov.

2.1 ZGODOVINA VIRTUALNIH RAČUNALNIKOV

Začetki virtualizacije segajo v 60. leta prejšnjega stoletja, ko je podjetje IBM razvilo prve sisteme za delitev zmogljivosti velikih računalnikov (mainframe). Namen teh rešitev je bil omogočiti več uporabnikom hkratno uporabo istega sistema, kar je povečalo učinkovitost in zmanjšalo stroške uporabe drage strojne opreme.

V naslednjih desetletjih se je razvoj virtualizacije upočasnil, saj so osebni računalniki postajali cenovno dostopnejši in zmogljivejši. Ponovni razcvet je tehnologija doživela v 90. letih in začetku 21. stoletja, ko so podjetja, kot je VMware, razvila sodobne rešitve za virtualizacijo na običajnih računalnikih. To je omogočilo širšo uporabo virtualnih računalnikov v podjetjih in kasneje tudi med posameznimi uporabniki.

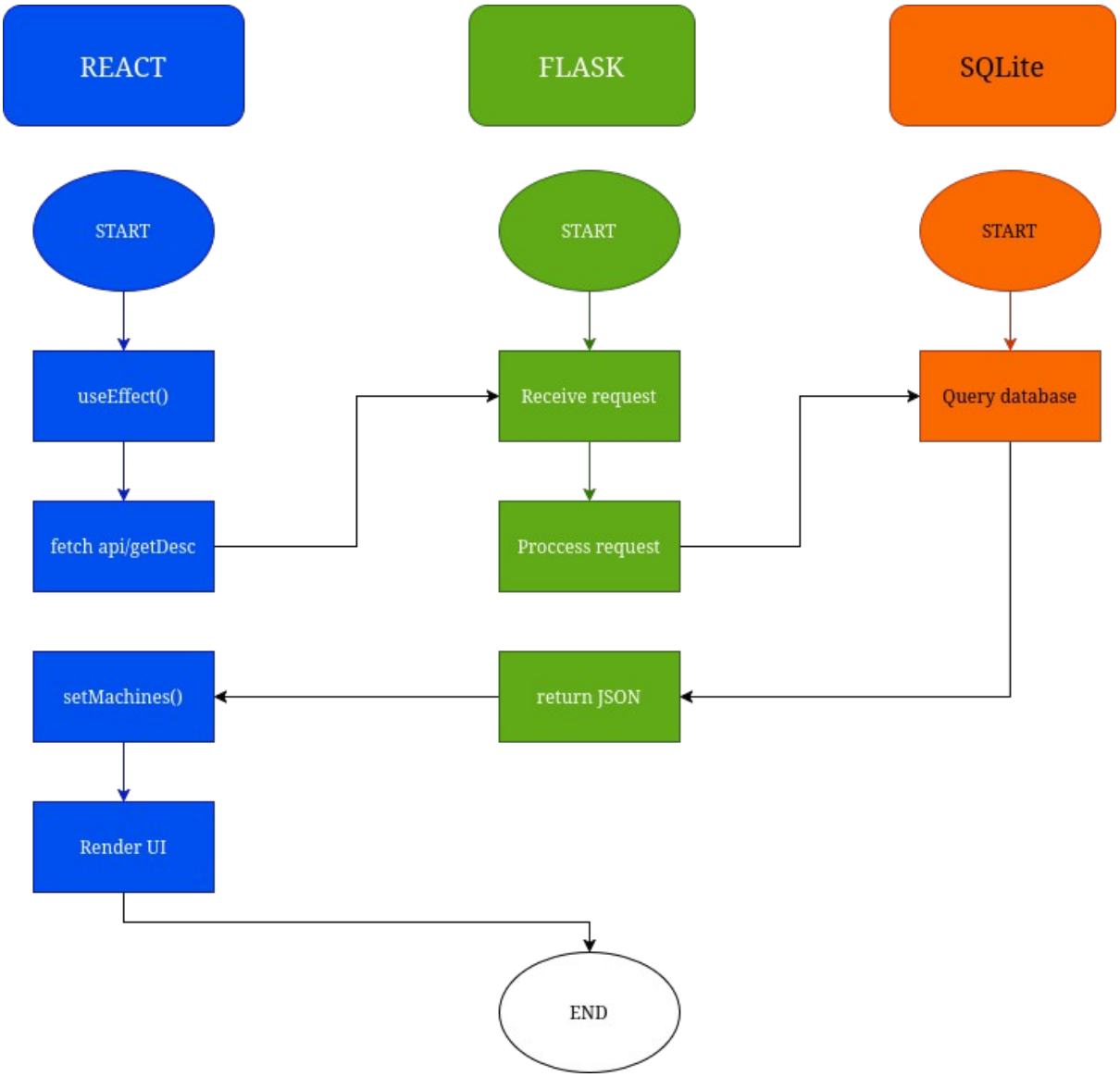
2.2 AKTUALNOST VIRTUALNIH RAČUNALNIKOV

Danes so virtualni računalniki zelo aktualni in pomembni, saj predstavljajo osnovo številnih sodobnih informacijskih sistemov. Uporabljajo se za razvoj programske opreme, testiranje, ter upravljanje strežniških okolij. Njihova uporaba omogoča večjo fleksibilnost, lažje upravljanje in boljšo izrabo strojnih virov.

3 POTEK DELA

Pri razvoju aplikacije sem uporabil tristopenjsko arhitekturo, kjer je sistem razdeljen na predstavitveni, aplikacijski in podatkovni sloj.

Predstavitveni sloj sem izdelal z uporabo React, ki omogoča uporabniški vmesnik. Aplikacijski sloj sem razvil z Flask, kjer sem obdeloval zahteve in povezal aplikacijo z libvirt za upravljanje virtualnih računalnikov. Podatkovni sloj predstavlja baza SQLite, ki skrbi za shranjevanje podatkov.



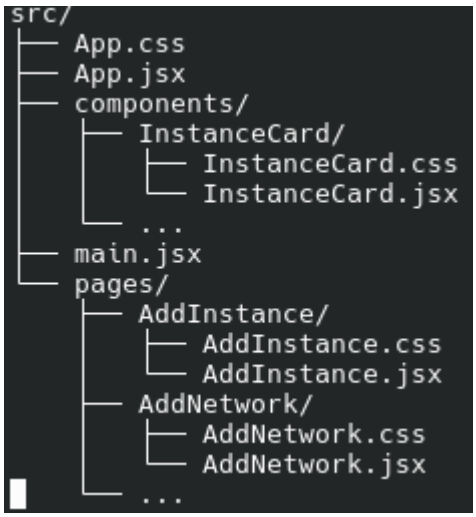
Slika 1: Prikaz poteka delovanja aplikacije

Vir: Lasten vir

3.1 RAZVOJ UPORABNIŠKEGA DELA

3.1.1 PODATKOVNA STRUKTURA

Podatkovno strukturo sem razdelil na komponente in strani. Vsaka komponenta vsebuje .jsx datoteko, ki poveže html ter javascript, zato ni potrebno dodatnih datotek za logiko strani in je posledično bolj pregledno. Poleg tega je v vsaki komeponenti datoteka css za oblikovanje. Stile, ki se nahajajo v več komponentah sem dodal v App.css (glavno komponento za oblikovanje) in s tem zmanjšal količino ponavljajoče se kode ter izboljšal preglednost. Interno usmerjanje (routing) sem definiral v App.jsx (glavno komponento).



Slika 2: Podatkovna struktura uporabniskega dela

Vir: Lasten vir

3.1.2 DELOVANJE

Potek delovanja posamezne strani deluje tako, da se ob zagonu strani kliče funkcija z useEffect. Ta pošlje zahtevo do aplikacijskega dela z ukazom fetch (npr.: fetch(http://localhost:5000/getDesc)). Nazaj dobi podatke v JSON formatu, ki jih nazaj pretvori v objekte. Če je proces uspešen se stanje obnovi z novimi podatki in stran se znova izriše(render-a). Ob uspešni pridobitvi podatkov aplikacijska stran vrne tudi uspešno sporočilo(message) ter statusno kodo 200 ali 201.

3.1.3 UPRAVLJANJE NAPAK

Pri neuspešnem procesu aplikacija prepozna dva tipa napak. Napako pri strežniški strani, ki jo zazna s ukazom if(!res.ok). Ukaz se sproži če je zahteva uspešna ampak je prišlo do napake pri strežniku. To preveri z statusno kodo (npr.: 404 Not Found, 500 Internal Server Error, 401 Unauthorized,…).Druga napaka pa se sproži med procesom pošiljanja. To se lahko zgodi če ne moremo doseči strežnika (npr.: DNS failure, CORS blocked request, No internet connection,…). To napako ujememo z ukazom catch(err).

```
const getXML = async () => {
  setError("");
  try {
    const store = await fetch("http://localhost:5000/api/storeDesc");
    const storeData = await store.json();

    if (!store.ok) {
      setError(storeData.error);
      return;
    }

    const res = await fetch("http://localhost:5000/api/getDesc");
    const data = await res.json();

    if (!res.ok) {
      setError(data.error);
    } else {
      setMachines(data);
    }
  } catch (err) {
    setError(err.message);
    console.error(err.message);
  }
};

useEffect(() => {
  getXML();
}, []);
```

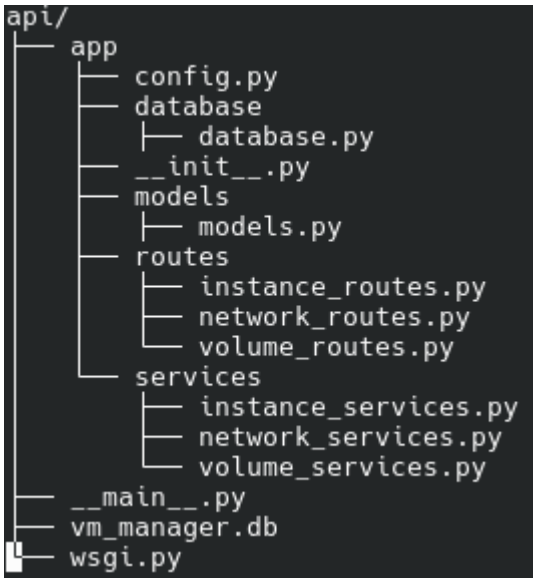
Slika 3: Primer kode za zahtevo podatkov

Vir: Lasten vir

3.2 RAZVOJ APLIKACIJSKEGA DELA

3.2.1 PODATKOVNA STRUKTURA

Aplikacijska stran se zažene preko datoteke __main__.py. Ta zažene aplikacijo v razvijalnem načinu s vklopljenim razhroščevalnikom(debuger). Config.py vsebuje konfiguracijo aplikacije. Med to spada pot do uporabniškega dela in do podatkovne baze, povezavo do podatkovne baze ter skrivnosti. __init__.py datoteka inicializira aplikacijo ter registrira načrte(blueprint). Pogonski sistem podatkovne baze je SQLite torej je baza v eni sami datoteki in sicer v vm_manager.py. ORM modeli za bazo so shranjeni v datoteki models.py. To so dejanske tabele, ki so definirane v jeziku python, kot razred. Povpraševalni stavki za bazo so shranjeni v datoteki database.py, le-te so klicani preko storitev. Storitve so ločene od usmerjevalniških poti, saj omogoča večjo preglednost. Vsaka storitev ima svoj usmerjevalnik, ki je nato vključen v datoteko __init__.py kot načrt(blueprint). To omogoča boljšo izolacijo med logiko ter večjo skalabilnost.



Slika 4: Podatkovna struktura aplikacijskega dela

Vir: Lasten vir

3.2.2 POVEZAVA Z BAZO

Za podatkovno bazo sem uporabil SQLite, saj omogoča enostavnejšo uporabo, hitrejše branje in vstavljanje podatkov, ter manjšo porabo prostora. Namenjene so predvsem manjšim aplikacijam, ki jih upravlja samo en uporabnik. Predvsem sem jih uporabil za shranjevanje podatkov o virtualnih računalnikih, saj je to veliko bolj varneje kot, da bi se zanašal na sejo libvirt povezave.

SQLite ima v flask-u zelo enostavno implementacijo. Najprej definiramo pogonski sistem podatkovne baze in nato z njim definiramo razred Session s katerim lahko posredno upravljamo bazo. Pred začetkom upravljanja s podatki moramo še ustvariti vse tabele, ki jih definiramo v datoteki models.py. Podatke upravljamo preko objekta, ki ga definiramo s razredom Session. Ob koncu vsake deklaracije pa moramo vsako sejo zaključit.

```python
#definiranje zagonskega sistema
engine = db.create_engine('sqlite:///vm_manager.db')
Session = sessionmaker(bind=engine)


class DatabaseServices:

    #funkcija, ki ustvari vse tabele
    @staticmethod
    def create_tables():
        try:
            Base.metadata.create_all(engine)
            return True
        except Exception as e:
            logger.error(f"Failed to create tables: {e}")
            return False

    @staticmethod
    def queryDesc():
        #definiranje objekta preko razreda Session
        session = Session()
        try:
            desc = session.query(Instances).all()
            arr = []
            for data in desc:
                arr.append({
                    "name": data.name,
                    "network": data.network,
                    "status": data.status,
                    "cpu": data.cpu,
                    "ram": data.ram,
                })
            return arr
        except Exception as e:
            #preverjanje napak
            logger.error(f"Error querying instances: {e}")
            raise
        finally:
            #zakljucek seje
            session.close()
```

Slika 5: Primer implementacije SQLite

Vir: Lasten vir

3.2.3 UPRAVLJANJE NAPAK

Aplikacija preverja različne tipe napak, kar je ključnega pomena pri razvijanju aplikacije ter v produkciji. Ob zaznavi napake aplikacija sprednjemu delu vrne sporočilo v JSON formatu, ter ustreznu statusno kodo. Napake lahko ločimo na logične in tiste, ki se sprožijo med procesom. Če želimo preverjati napake med procesom moramo gnezditi tisti del kode v try, napako pa lovimo z ukazom except.

```
def start(self, name):
    conn = None
    instance = DatabaseServices.getInstance(name)
    if not instance:
        return jsonify({"error": "instance not found"}), 404

    try:
        conn = libvirt.open(instance.uri)
        if not conn:
            return jsonify({"error": "cannot open libvirt connection"}), 500

        dom = conn.lookupByName(name)
        if not dom:
            return jsonify({"error": "domain not found"}), 404

        if dom.isActive():
            return jsonify({"message": "machine is already running"}), 400

        dom.create()

        return jsonify({"message": "instance started successfully"}), 200

    except libvirt.libvirtError as e:
        return jsonify({"error":  str(e)})
    except Exception as e:
        return jsonify({"error": str(e)})

    finally:
        if conn:
            conn.close()
```

Slika 6: Primer flask kode

Vir: Lasten vir

3.2.4 ZAGON APLIKACIJE

Aplikacijo lahko zaženemo na dva različna načina in sicer na razvijalski način ter na produkcijski način. Razvijalski način zaženemo preko datoteke __main__.py in omogoča vgrajen strežnik werkzeug, razhroščevalnik ter samodejno osveževanje strani(hot reload). Namenjen je predvsem lokalnemu testiranju in razvijanje. Produkcijski način pa zaženemo preko datoteke wsgi.py s strežnikom Gunicorn. To omogoča obdelovanje več zahtev hkrati nima pa razhroščevalnika, saj je namenjen za produkcijo. Te dve datoteki sta od znotraj povsem enaki.

```
import os
import sys
from app import create_app

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

app = create_app()
```

Slika 7: Primer zagona aplikacije

Vir: Lasten vir

4 Funkcije

4.1 FUNKCIJE VEZANE NA VIRTUALNE RAČUNALNIKE

Osnovne funkcije, ki jih aplikacija ponuja za virtualne računalnike so pogled, dodajanje in odstranjevanje, zagon, ustavitev in izklop. Omogoča pa še tudi kreiranje s že obstoječega virtualnega računalnika, ter shranjevanje(snapshot).

Pogled deluje tako, da ob začetku poišče in shrani vse virtualne računalnike v bazo in nato iz baze pridobi podatke. Shranjevanje v bazo je ključnega pomena, ker če bi stran direktno pridobivala podatke iz libvirt seje, bi lahko prišlo do prekinitve seje in stran ne bi več prikazovala podatkov.

Kreiranje virtualnega računalnika deluje tako, da najprej pridobimo ustrezne podatke preko čelne strani, ki jih pošljemo na strežniško stran. Na strežniški strani preverimo če podatki vključujejo pot do iso datoteke, če ne uporabimo disk za nalagalnik operacijskega sistema. Nato še ustvarimo preostalek xml in z njim definiramo virtualni računalnik.

Pri odstranjevanju najprej odstranimo datoteko za shranjevanje tako da, poiščemo pot do datoteke, ki jo najdemo preko definicije virtualnega računalnika v xml kodi in shrambo izbrišemo. Ob koncu pa še odstranimo defnicijo virtualnega računalnika.

Zagon, ustavitev in izklop deluje po skoraj istem postopku. Najprej preverimo če je virtualni računalnik izklopljen ali vklopljen in nato ustrezno spremenimo njegov status.

Kreiranje virtualnega računalnika s že obstoječega(create from template) deluje tako, da poišče podatke že obstoječega in definira novega. Edini podatek, ki ga mora uporabnik podati je drugačno ime. Avtomatsko pa se še doda nova qcow2 datoteka za shranjevanje. Ta metoda ne potrebuje iso datoteke saj kopira že obstoječo datoteko v novo in preko nje naloži operacijski sistem.

4.2 FUNKCIJE VEZANE NA ZBIRKO PODATKOV

Aplikacije omogoča pogled nad podatkovno zbirko(pool), ter na diske(volumes). Lahko pa tudi kreiramo nove diske v določeni podatkovni zbirki.

Pogled nad podatkovno zbirko je preprosta funkcija, ki vrača ime, število diskov, velikost in porabljen prostor določene zbirke. Ob kliku na zbirko se izpišejo vsi diski v tisti zbirki. To deluje tako da v url dodamo atribut ime in preko njega poiščemo vse diske.

Aplikacija omogoča tudi kreiranje novega diska. Funkcija prejme ime diska in capaciteto, ki jo poda uporabnik, ter ime zbirke. Nato definira nov xml in kreaira disk v določeni zbirki.

4.3 FUNKCIJE VEZANE NA OMREŽJE

Aplikacija omogoča dodajanje odjemalca za ssh povezavo. Na začetku moramo v .shh/config datoteki definirati novo povezavo. Nato preko aplikacije dodamo ime nove povezave. To omogoča upravljanje virtualnih računalnikov, ki so na drugem računalniku.

5 ZAKLJUČEK

Aplikacija omogoča osnovne funkcionalnosti za upravljanje virtualnih računalnikov. Implementacjia na spletu mi je omogočila kreiranje sodobnejšega uporabniškega vmesnika ter večjo portabilnost med distribucijami. Težave, ki jih predstavlja tak sistem so manjša optimizacija in hitrost. manjkajo pa tudi napredne funkcije ter implementacija vnc-ja. S tem projektom sem se naučil veliko o samem delovanju virtualnih računalnikov, kako razvijemo večjo aplikacijo in kako jo spravimo v produkcijo.

6 VIRI IN LITERATURA

1. Libvirt Project. Libvirt (online). 2026. (citirano 4. 3. 2026). Dostopno na naslovu: https://libvirt.org/
2. Meta (Facebook). React – A JavaScript library for building user interfaces (online). 2026. (citirano 4. 3. 2026). Dostopno na naslovu: https://reactjs.org/
3. Python Software Foundation. Python Programming Language (online). 2026. (citirano 4. 3. 2026). Dostopno na naslovu: https://python.org
4. Pallets Projects. Flask – A lightweight WSGI web application framework (online). 2026. (citirano 4. 3. 2026). Dostopno na naslovu: https://flask.palletsprojects.com/